

Row-Level Security

(RLS)

Seguridad a nivel de fila en PostgreSQL

Ing. Ramsés Camas Nájera • UPCh 2026



RLS

CREATE POLICY

USING

WITH CHECK

Multi-tenant

¿Qué veremos hoy?

01

El problema que GRANT no puede resolver

Por qué necesitamos seguridad a nivel de fila

02

GRANT vs RLS — conceptos y diferencias

Cuándo usar uno, cuándo el otro, cuándo ambos

03

Habilitar RLS y CREATE POLICY

ALTER TABLE, sintaxis completa de políticas

04

USING vs WITH CHECK

Filtro de lectura vs validación de escritura

05

Políticas por rol, múltiples políticas

TO role, políticas que se combinan con OR

06

FORCE ROW LEVEL SECURITY

Que aplique incluso al dueño de la tabla

07

current_setting() para multi-tenant

Pasar el tenant_id como variable de sesión

08

Casos prácticos + Errores + Ejercicio

Empleados, hospital, multi-tenant desde cero

El Problema que GRANT no puede resolver

GRANT controla tablas enteras — no filas individuales

Situación real

"El enfermero_juan tiene SELECT en historial_clinico... pero puede ver el historial de TODOS los pacientes, no solo los suyos."

```
-- Esto es lo que GRANT puede hacer:  
GRANT SELECT ON historial_clinico TO enfermero;  
-- Juan puede leer toda la tabla ✓... pero ve TODOS los pacientes ✗  
  
-- Lo que queremos:  
-- Juan solo ve filas donde empleado_id = juan  
-- Dra. López solo ve filas donde medico_id = dr_lopez  
-- Cada empresa solo ve SUS propios datos  
  
-- GRANT no puede hacer esto. Necesitamos RLS.
```

✓ Con RLS

Cada usuario ve exactamente las filas que le corresponden, automáticamente, sin lógica en la app.

✗ Sin RLS

Debes filtrar con WHERE en cada query. Un error en la app expone datos de todos.

GRANT vs RLS — ¿Cuándo usar cada uno?

No se reemplazan — se complementan

Característica	GRANT	RLS
¿Qué controla?	Acceso a objetos (tabla, columna, función)	Acceso a filas dentro de una tabla
Nivel de granularidad	Tabla / Columna	Fila individual
¿Dónde vive?	En el objeto (tabla, schema)	En la tabla como política
¿Quién lo usa?	Todos los proyectos	Multi-tenant, datos personales, auditoría por usuario
¿Se pueden combinar?	Sí — GRANT es el primer filtro	Sí — RLS es el segundo filtro
Sin este → error	permission denied for table	El usuario ve filas de otros

 Regla: GRANT define QUÉ TABLAS puede ver un rol. RLS define QUÉ FILAS dentro de esas tablas.

Paso 1: Habilitar RLS en una Tabla

RLS está desactivado por defecto en todas las tablas

```
-- Habilitar RLS en la tabla
ALTER TABLE historial_clinico ENABLE ROW LEVEL SECURITY;

-- Deshabilitar (vuelve al comportamiento normal)
ALTER TABLE historial_clinico DISABLE ROW LEVEL SECURITY;

-- FORCE: aplica RLS incluso al dueño de la tabla (ver slide 12)
ALTER TABLE historial_clinico FORCE ROW LEVEL SECURITY;

-- Verificar si RLS está activo
SELECT tablename, rowsecurity, forcerowsecurity
FROM pg_tables
WHERE schemaname = 'public'
   AND tablename = 'historial_clinico';

-- También en information_schema
SELECT table_name, row_security
FROM information_schema.tables
WHERE table_schema = 'public';
```



Comportamiento por defecto

Con RLS habilitado pero SIN ninguna política creada:

→ Ningún usuario (excepto el dueño y superusers) puede ver ninguna fila.

Es una negación total por defecto.



Flujo correcto

1. ENABLE ROW LEVEL SECURITY
2. Crear políticas con CREATE POLICY
3. Probar con SET ROLE
4. Verificar en pg_policies

pg_tables útil

rowsecurity = true → RLS activo
forcerowsecurity = true → aplica al dueño también

CREATE POLICY — Sintaxis Completa

La instrucción central de RLS

```
CREATE POLICY nombre_politica
  ON nombre_tabla
  [AS PERMISSIVE | RESTRICTIVE]      -- PERMISSIVE por defecto
  [FOR ALL | SELECT | INSERT | UPDATE | DELETE]
  [TO rol_o_usuario]                -- PUBLIC si se omite
  [USING (expresion_filtro)]         -- ¿qué filas son visibles?
  [WITH CHECK (expresion_validacion)]; -- ¿qué filas se pueden escribir?
```

-- Ejemplo: cada empleado solo ve su propio registro

```
CREATE POLICY ver_mi_expediente
  ON empleados FOR SELECT TO empleado_rol
  USING (id = current_user::INT);
```

-- Ejemplo: solo filas activas son visibles

```
CREATE POLICY solo_activos
  ON productos FOR SELECT
  USING (activo = TRUE);
```

-- Ejemplo: cada usuario ve y edita solo lo suyo

```
CREATE POLICY mis_pedidos
  ON pedidos FOR ALL
  USING (usuario_id = current_user::INT)
  WITH CHECK(usuario_id = current_user::INT);
```

USING — El Filtro de Lectura

Se aplica a SELECT, UPDATE y DELETE — define qué filas son visibles

USING (expresión) → se comporta como un WHERE invisible aplicado automáticamente a cada query.

```
-- Tabla: notas_medicas (id, paciente_id, medico_id, nota, fecha)

-- Política: el médico solo ve las notas que él escribió
CREATE POLICY notas_del_medico
  ON notas_medicas FOR SELECT TO medico
  USING (medico_id = current_user::INT);

-- Cuando dra_lopez (id=5) hace SELECT * FROM notas_medicas
-- PostgreSQL ejecuta internamente:
-- SELECT * FROM notas_medicas WHERE medico_id = 5;
-- dra_lopez NUNCA ve las notas de otros médicos

-- Aplica también en UPDATE y DELETE:
CREATE POLICY actualizar_mis_notas
  ON notas_medicas FOR UPDATE TO medico
  USING (medico_id = current_user::INT);
-- Si intenta UPDATE en nota de otro médico
-- → 0 rows affected (no error, solo silencio)
```

WITH CHECK — Validación de Escritura

Se aplica a INSERT y UPDATE — valida que la fila escrita sea válida

WITH CHECK (expresión) → valida cada fila que se intenta insertar o actualizar. Si la expresión es FALSE → ERROR.

```
-- Política: un empleado solo puede crear notas con SU propio medico_id
CREATE POLICY insertar_mis_notas
  ON notas_medicas
  FOR INSERT
  TO medico
  WITH CHECK (medico_id = current_user::INT);

-- Si dra_lopez intenta:
INSERT INTO notas_medicas(paciente_id, medico_id, nota)
VALUES (10, 99, 'Nota falsa con otro médico');
-- ERROR: new row violates row-level security policy ✓

-- Política completa SELECT + INSERT + UPDATE:
CREATE POLICY mis_notas_completo
  ON notas_medicas
  FOR ALL
  TO medico
  USING      (medico_id = current_user::INT)  -- filtro lectura
  WITH CHECK(medico_id = current_user::INT); -- validación escritura
```

USING falla → la fila no existe (silencio)

WITH CHECK falla → ERROR explícito de violación

USING vs WITH CHECK — Cuándo aplica cada uno

Comando	USING	WITH CHECK	¿Qué hace?
SELECT	USING ✓	—	¿Qué filas puedo leer?
INSERT	—	WITH CHECK ✓	¿Puedo insertar esta fila?
UPDATE	USING ✓	WITH CHECK ✓	¿Puedo ver Y guardar esta fila?
DELETE	USING ✓	—	¿Puedo eliminar esta fila?
ALL	USING ✓	WITH CHECK ✓	Cubre todos los comandos

```
-- Recomendación: para UPDATE usa AMBOS
CREATE POLICY actualizar_pedido
  ON pedidos FOR UPDATE TO cliente
  USING      (cliente_id = current_user::INT) -- solo VE sus pedidos
  WITH CHECK(cliente_id = current_user::INT); -- solo GUARDA en sus pedidos
-- Sin WITH CHECK en UPDATE: podría reasignar la fila a otro cliente
```

PERMISSIVE vs RESTRICTIVE

Cómo se combinan múltiples políticas sobre la misma tabla

Cuando hay varias políticas: PERMISSIVE se combinan con OR / RESTRICTIVE se combinan con AND (filtro adicional)

```
-- PERMISSIVE (por defecto): la fila es visible si CUALQUIER política lo permite
CREATE POLICY ver_propios AS PERMISSIVE
  ON historial_clinico FOR SELECT TO medico
  USING (medico_id = current_user::INT);

CREATE POLICY ver_urgentes AS PERMISSIVE
  ON historial_clinico FOR SELECT TO medico
  USING (urgente = TRUE);
-- Resultado: el médico ve sus filas OR las urgentes

-- RESTRICTIVE: la fila solo es visible si TODAS las políticas lo permiten
CREATE POLICY solo_activos AS RESTRICTIVE
  ON historial_clinico FOR SELECT TO medico
  USING (activo = TRUE);
-- Ahora el médico ve: (sus filas OR urgentes) AND activo=TRUE

-- Fórmula final:
-- filas visibles = (PERMISSIVE_1 OR PERMISSIVE_2 OR ...) AND (RESTRICTIVE_1 AND RESTRICTIVE_2 ...)
```

Políticas por Rol — TO role

Cada rol puede tener sus propias reglas de acceso a filas

```
-- Tabla: expedientes_medicos
-- Columnas: id, paciente_id, medico_id, enfermero_id, datos,
confidencial

-- Política para el rol médico: ve sus expedientes y los urgentes
CREATE POLICY medico_ve_suyos
  ON expedientes_medicos FOR SELECT TO medico
  USING (medico_id = current_user::INT);

-- Política para el rol enfermero: ve expedientes de su planta (no
los confidenciales)
CREATE POLICY enfermero_ve_suyos
  ON expedientes_medicos FOR SELECT TO enfermero
  USING (enfermero_id = current_user::INT
        AND confidencial = FALSE);

-- Política para administrador: ve todo
CREATE POLICY admin_ve_todo
  ON expedientes_medicos FOR ALL TO administrador
  USING (TRUE);
-- USING(TRUE) = sin filtro = acceso total a todas las filas

-- Los roles sin política → ven CERO filas (comportamiento por
defecto)
```

TO PUBLIC

Si omites TO, la política aplica a TODOS los roles:

```
CREATE POLICY solo_activos
  ON productos
  USING (activo = TRUE);
```

Todo el mundo ve solo productos activos.

USING (TRUE)

Dar acceso total a una tabla con RLS habilitado:

```
CREATE POLICY acceso_total
  ON tabla FOR ALL
  TO admin_rol
  USING (TRUE)
  WITH CHECK (TRUE);
```

Sin política → sin datos

Con RLS activo, un rol sin política asignada obtiene 0 filas en SELECT y ERROR en INSERT/UPDATE.

FORCE ROW LEVEL SECURITY

Por defecto, el dueño de la tabla ignora RLS — FORCE lo obliga a respetarlo

 **El dueño de la tabla (owner) IGNORA RLS por defecto — puede ver todas las filas aunque haya políticas activas.**

```
-- Verificar: ¿quién es el dueño de la tabla?
SELECT tableowner FROM pg_tables
WHERE tablename = 'historial_clinico';

-- Sin FORCE: el dueño ve todo
ALTER TABLE historial_clinico ENABLE ROW LEVEL SECURITY;
-- dba_user (dueño) → SELECT * → ve TODAS las filas ← problema

-- Con FORCE: el dueño también debe cumplir las políticas
ALTER TABLE historial_clinico FORCE ROW LEVEL SECURITY;
-- dba_user → SELECT * → solo ve las filas que sus políticas permiten

-- Nota: SUPERUSER siempre ignora RLS incluso con FORCE
-- Para aplicar RLS a superusers, deben usar SET ROLE explícitamente

-- Quitar FORCE
ALTER TABLE historial_clinico NO FORCE ROW LEVEL SECURITY;

-- Ver estado actual
SELECT tablename, rowsecurity, forcerowsecurity
FROM pg_tables WHERE tablename = 'historial_clinico';
```

Funciones Útiles dentro de las Políticas

Identificar al usuario activo en la expresión USING

```
-- current_user: el usuario que ejecuta la query (cambia con SET
ROLE)
CREATE POLICY mis_ventas
  ON ventas FOR SELECT
  USING (vendedor = current_user);

-- session_user: el usuario de login original (no cambia con SET
ROLE)
CREATE POLICY log_acceso
  ON logs FOR INSERT
  WITH CHECK (quien = session_user);

-- Cuando el ID es entero en vez de nombre:
CREATE POLICY mi_expediente
  ON empleados FOR SELECT
  USING (empleado_id = (
    SELECT id FROM usuarios WHERE username = current_user
  ));

-- Combinando con otras condiciones:
CREATE POLICY ver_propios_o_publicos
  ON documentos FOR SELECT
  USING (
    propietario = current_user
    OR visibilidad = 'publico'
  );
```

current_user vs session_user

current_user = activo ahora
→ cambia con SET ROLE
→ usar para filtros de datos

session_user = quien hizo login
→ nunca cambia
→ usar para logs y auditoría

Otras funciones útiles

inet_client_addr()
→ IP del cliente

current_timestamp
→ para políticas con fecha

current_setting('app.user_id')
→ parámetro de sesión
(ver slide multi-tenant)

Ojo con tipos de dato

Si tu columna es INTEGER y current_user devuelve
TEXT:

current_user::INTEGER

o usa una función auxiliar que mapee username → id

current_setting() — La clave del Multi-Tenant

Pasar el tenant_id como variable de sesión en lugar de usar el username

En sistemas multi-tenant, todos los usuarios se conectan con el mismo role de app. La variable de sesión identifica a qué empresa pertenecen.

```
-- 1. La app establece el tenant al inicio de cada sesión
SET app.current_tenant_id = '42'; -- empresa con ID 42

-- 2. La política usa current_setting() para leerlo
CREATE POLICY tenant_isolation ON pedidos FOR ALL
USING (
    empresa_id = current_setting('app.current_tenant_id')::INT
)
WITH CHECK (
    empresa_id = current_setting('app.current_tenant_id')::INT
);

-- 3. Verificación desde la app
SELECT current_setting('app.current_tenant_id'); -- '42'

-- 4. Con valor por defecto (evita error si no está seteado)
USING (
    empresa_id = NULLIF(
        current_setting('app.current_tenant_id', TRUE), ''
    )::INT
);
```

¿Por qué no usar current_user?

En multi-tenant, todos los microservicios comparten un mismo usuario de BD:

app_user → empresa_A
app_user → empresa_B
app_user → empresa_C

El usuario es el mismo. El tenant_id es el diferenciador.



Precaución de seguridad

Si la app no hace SET antes de la query, el valor queda del tenant anterior.

Siempre usa SET al inicio de cada transacción o conexión, no una sola vez.

Gestionar Políticas — ALTER, DROP, ver activas

```
-- Ver todas las políticas activas en el schema
SELECT schemaname, tablename, policyname,
       permissive, roles, cmd,
       qual AS using_expr, with_check
FROM pg_policies
WHERE schemaname = 'public'
ORDER BY tablename, policyname;

-- Ver políticas de una tabla específica
SELECT policyname, cmd, roles, qual, with_check
FROM pg_policies
WHERE tablename = 'historial_clinico';

-- Modificar una política existente
ALTER POLICY mis_notas ON notas_medicas
  USING (medico_id = current_user::INT AND activo = TRUE);

-- Renombrar una política
ALTER POLICY mis_notas ON notas_medicas
  RENAME TO notas_del_medico_activo;

-- Eliminar una política
DROP POLICY notas_del_medico_activo ON notas_medicas;

-- Deshabilitar RLS sin borrar las políticas
-- (las políticas quedan guardadas pero inactivas)
ALTER TABLE notas_medicas DISABLE ROW LEVEL SECURITY;
```

Debugging de RLS — Cómo probar tus políticas

SET ROLE para simular cada usuario y verificar qué ve

```
-- Paso 1: verifica que RLS esté activo
SELECT tablename, rowsecurity FROM pg_tables
WHERE tablename = 'pedidos';

-- Paso 2: verifica qué políticas existen
SELECT policyname, cmd, roles, qual
FROM pg_policies WHERE tablename = 'pedidos';

-- Paso 3: simula ser otro usuario
SET ROLE juan;                -- juan tiene rol 'cliente'
SELECT * FROM pedidos;        -- ¿qué filas ve juan?
RESET ROLE;

SET ROLE ana;                 -- ana tiene rol 'vendedor'
SELECT * FROM pedidos;        -- ¿qué ve ana?
RESET ROLE;

-- Paso 4: si ves 0 filas y no debería:
-- a) ¿Tiene GRANT SELECT en la tabla?
-- b) ¿La política tiene USING correcto?
-- c) ¿Los valores coinciden? (tipos de dato)

-- Paso 5: ver qué evalúa la política para el usuario actual
EXPLAIN SELECT * FROM pedidos;
-- El Filter: en el plan muestra la condición de RLS aplicada
```

Checklist de debugging

- ☐ RLS habilitado en la tabla?
- ☐ Política creada para ese rol?
- ☐ GRANT SELECT dado también?
- ☐ ¿Tipo de dato correcto en USING?
- ☐ ¿Usuario tiene el rol correcto?
- ☐ ¿USING(TRUE) para el admin?

Error frecuente: tipos

ERROR: operator does not exist: integer = text

La política tiene:
USING (user_id = current_user)

Pero user_id es INTEGER y current_user es TEXT.

Fix:
USING (user_id = (
 SELECT id FROM users
 WHERE uname = current_user
))

Caso 1: Empleados Solo Ven Su Propio Expediente

-- Tabla de empleados

CREATE TABLE empleados (

id SERIAL PRIMARY KEY,

username TEXT UNIQUE NOT NULL, -- igual al nombre de usuario en BD

nombre TEXT,

salario NUMERIC,

departamento TEXT

);

-- Habilitar RLS

ALTER TABLE empleados ENABLE ROW LEVEL SECURITY;

ALTER TABLE empleados FORCE ROW LEVEL SECURITY;

-- Política: cada empleado solo ve su propia fila

CREATE POLICY ver_mi_registro

ON empleados

FOR SELECT

TO empleado_rol

USING (username = current_user);

-- Política: RRHH ve todo

CREATE POLICY rrhh_ve_todo

ON empleados

FOR ALL

TO rrhh

USING (TRUE)

WITH CHECK (TRUE);

-- Prueba

SET ROLE juan;

SELECT * FROM empleados; -- solo ve su fila

SELECT COUNT(*) FROM empleados; Camas devuelve 1

Caso 2: Hospital — Acceso por Rol y Confidencialidad

Múltiples políticas, múltiples roles

```
CREATE TABLE historial_clinico (  
  id SERIAL PRIMARY KEY,  
  paciente_id INT,  
  medico_id INT, -- id del médico que lo creó  
  enfermero_id INT,  
  diagnostico TEXT,  
  confidencial BOOLEAN DEFAULT FALSE  
);  
  
ALTER TABLE historial_clinico ENABLE ROW LEVEL SECURITY;  
  
-- Médico: ve los historiales donde él es el médico tratante  
CREATE POLICY medico_sus_pacientes  
  ON historial_clinico FOR SELECT TO medico  
  USING (medico_id = (  
    SELECT id FROM staff WHERE username = current_user  
  ));  
  
-- Enfermero: ve historiales de su planta, excepto confidenciales  
CREATE POLICY enfermero_no_confidencial  
  ON historial_clinico FOR SELECT TO enfermero  
  USING (  
    enfermero_id = (SELECT id FROM staff WHERE username = current_user)  
    AND confidencial = FALSE  
  );  
  
-- Director médico: ve todo  
CREATE POLICY director_acceso_total  
  ON historial_clinico FOR ALL TO director  
  USING (TRUE) WITH CHECK (TRUE);
```

Caso 3: Sistema Multi-Tenant Completo

Una sola BD, múltiples empresas — cada una aislada completamente

```
-- Tabla compartida entre todas las empresas
CREATE TABLE pedidos (
  id          SERIAL PRIMARY KEY,
  empresa_id  INT NOT NULL,
  producto    TEXT,
  cantidad    INT,
  total       NUMERIC
);

ALTER TABLE pedidos ENABLE ROW LEVEL SECURITY;

-- Política de aislamiento por tenant
CREATE POLICY tenant_isolation
  ON pedidos FOR ALL
  USING (
    empresa_id = current_setting('app.tenant_id')::INT
  )
  WITH CHECK (
    empresa_id = current_setting('app.tenant_id')::INT
  );

-- La app backend hace esto al inicio de cada request:
BEGIN;
  SET LOCAL app.tenant_id = '42'; -- empresa actual del request
  SELECT * FROM pedidos;           -- automáticamente filtra empresa_id=42
  INSERT INTO pedidos(empresa_id, producto, cantidad, total)
  VALUES (42, 'Laptop', 3, 45000); -- WITH CHECK valida empresa_id=42 ✓
COMMIT;

-- SET LOCAL expira al final de la transacción ← más seguro que SET
```

SET LOCAL vs SET — Seguridad en Multi-Tenant

La diferencia crítica para no filtrar datos entre requests

SET — Persiste en la conexión

```
BEGIN;
  SET app.tenant_id = '42';
  SELECT * FROM pedidos;
COMMIT;

-- ⚠ Problema:
-- La variable SIGUE siendo 42
-- en el próximo request!

BEGIN;
  -- Olvidamos el SET
  SELECT * FROM pedidos;
  -- ← ve pedidos de empresa 42
  -- aunque sea empresa 99!
COMMIT;
```

✗ Riesgo de fuga de datos

SET LOCAL — Solo en la transacción

```
BEGIN;
  SET LOCAL app.tenant_id='42';
  SELECT * FROM pedidos;
COMMIT;
-- Al hacer COMMIT, el valor
-- se resetea automáticamente

BEGIN;
  -- Si olvidamos el SET LOCAL
  SELECT * FROM pedidos;
  -- ← ERROR o 0 filas
  -- (el setting ya no existe)
COMMIT;
```

✓ Aislamiento garantizado por transacción

RLS + Vistas — Combinación Poderosa

Las vistas heredan las políticas de RLS de las tablas base

```
-- La vista aplica el RLS de la tabla historial_clinico
CREATE VIEW mi_historial AS
  SELECT id, paciente_id, diagnostico, fecha
  FROM historial_clinico;
-- Los médicos que hagan SELECT en mi_historial
-- automáticamente respetan las políticas de la tabla base

-- SECURITY DEFINER: la vista se ejecuta con permisos del creador
CREATE VIEW resumen_publico
  WITH (security_invoker = FALSE) -- = SECURITY DEFINER
AS
  SELECT id, nombre FROM empleados;
-- Útil cuando quieres dar acceso a una vista
-- sin dar acceso directo a la tabla

-- SECURITY INVOKER (recomendado con RLS): usa permisos de quien
-- consulta
CREATE VIEW mi_resumen
  WITH (security_invoker = TRUE)
AS
  SELECT * FROM historial_clinico;
-- Aplica las políticas RLS del usuario que hace SELECT en la vista

-- Verificar
SELECT viewname, definition
FROM pg_views
WHERE viewname = 'mi_historial';
```

Security Invoker vs Definer

INVOKER (default en PG 15+):

- RLS del usuario que consulta
- más seguro con RLS

DEFINER:

- RLS del creador de la vista
- puede bypassar políticas
- **usar con cuidado**

Caso de uso: vista pública con tabla privada

La tabla empleados tiene RLS — nadie puede ver salarios.

La vista empleados_directorio muestra solo (nombre, depto).

Das GRANT SELECT en la VISTA, no en la tabla.

Los usuarios ven el directorio pero jamás acceden a la tabla base directamente.

Errores Comunes con RLS — y sus Fixes

Los 5 errores que todos cometen la primera vez

Veo 0 filas aunque debería ver algo

→ RLS activo pero ninguna política aplica a mi rol

Fix: `CREATE POLICY ... TO mi_rol USING(TRUE);` — o revisa qué políticas existen en `pg_policies`

ERROR: operator does not exist: integer = text

→ Tipo de dato incompatible en USING (columna INT, current_user TEXT)

Fix: Usa un subquery: `USING(col_id = (SELECT id FROM users WHERE uname = current_user))`

El dueño de la tabla ve todas las filas

→ El owner ignora RLS por defecto

Fix: `ALTER TABLE tabla FORCE ROW LEVEL SECURITY;`

WITH CHECK violation en UPDATE correcto

→ USING filtra filas visibles pero WITH CHECK valida el nuevo estado — deben ser consistentes

Fix: Asegúrate que USING y WITH CHECK usen la misma condición en el `UPDATE`

Datos del tenant anterior aparecen en nuevo request

→ Usaste SET en lugar de SET LOCAL — el setting persiste en la conexión

Fix: Usa `SET LOCAL app.tenant_id = '...'` dentro de `BEGIN/COMMIT`

Buenas Prácticas con RLS

Siempre FORCE para tablas sensibles

En tablas con datos personales o financieros, usa FORCE ROW LEVEL SECURITY para que el owner también respete las políticas.

Prueba con SET ROLE antes de producción

Simula cada rol con SET ROLE y verifica que ve exactamente lo que debe ver. Documenta los casos de prueba.

SET LOCAL, no SET, en multi-tenant

SET LOCAL garantiza que el tenant_id se resetea al final de cada transacción. SET puede dejar datos cruzados entre requests.

Agrega índice en la columna de filtro

La columna que usas en USING (ej: empresa_id, user_id) debe tener un índice. Sin índice, RLS se vuelve un full scan por cada query.

Nombra políticas descriptivamente

BIEN: cliente_ve_sus_pedidos
MAL: politica1, pol_select
El nombre debe decir QUÉ hace y QUIÉN puede.

RLS no reemplaza a GRANT

RLS es el segundo nivel. Siempre necesitas GRANT CONNECT + GRANT USAGE + GRANT SELECT como base. RLS filtra, no autoriza.

Rendimiento — RLS e Índices

Sin índice en la columna del filtro, RLS puede destruir el performance

⚠ **USING(empresa_id = ...)** hace un filtro en CADA query. Sin índice en empresa_id → full scan de toda la tabla.

```
-- ✗ Sin índice — SLOW
CREATE TABLE pedidos (
  id          SERIAL PRIMARY KEY,
  empresa_id INT NOT NULL,
  datos       TEXT
);
ALTER TABLE pedidos ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant ON pedidos
  USING (empresa_id = current_setting('app.tenant_id')::INT);
-- Cada SELECT hace Seq Scan de toda la tabla ← lento con millones de
filas

-- ✓ Con índice — FAST
CREATE INDEX idx_pedidos_empresa
  ON pedidos(empresa_id);
-- Ahora el planificador usa Index Scan directo ← rápido

-- Verificar que el índice se usa
EXPLAIN ANALYZE
SELECT * FROM pedidos;
-- Busca: Index Scan using idx_pedidos_empresa
-- Evita: Seq Scan (señal de que falta el índice)
```

Columnas que necesitan índice

Las típicas en USING:

- empresa_id (multi-tenant)
- user_id / empleado_id
- medico_id, paciente_id
- tenant_id

Regla: si está en USING → ponle índice.

Índice parcial opcional

Si solo filtras activos:

```
CREATE INDEX idx_activos
  ON pedidos(empresa_id)
  WHERE activo = TRUE;
```

Más pequeño y rápido.

Resumen: Flujo Completo para Implementar RLS

El orden correcto de principio a fin

1	Diseña quién debe ver qué filas — antes de escribir SQL
2	GRANT los permisos base (CONNECT, USAGE, SELECT/INSERT/UPDATE)
3	ALTER TABLE tabla ENABLE ROW LEVEL SECURITY
4	ALTER TABLE tabla FORCE ROW LEVEL SECURITY (si es sensible)
5	CREATE POLICY para cada rol con USING y/o WITH CHECK
6	Crea un índice en la columna usada en USING
7	Prueba con SET ROLE para cada rol — verifica qué ve cada uno
8	En multi-tenant: usa SET LOCAL dentro de BEGIN/COMMIT
9	Documenta cada política en pg_description o comentarios

Ejercicio en Clase

35 minutos — implementa RLS desde cero

Escenario: Plataforma Educativa Online

Tablas: cursos · lecciones · calificaciones · comentarios · pagos

Roles: estudiante, instructor, admin — Users: juan (estudiante), prof_ana (instructor), root_admin

A Habilita RLS con FORCE en: calificaciones, pagos

B estudiante solo ve sus propias calificaciones (alumno_id = id del usuario)

C instructor solo ve calificaciones de sus cursos (curso.instructor_id = usuario)

D estudiante ve y crea sus propios comentarios; no puede modificar los de otros

E estudiante solo ve sus propios pagos (confidencial)

F instructor ve todos los cursos pero solo puede editar los que él creó

G admin: USING(TRUE) en todas las tablas

H Prueba con SET ROLE juan, prof_ana, root_admin y reporta qué ve cada uno



RLS = Seguridad invisible

"El mejor filtro de seguridad es el que el usuario ni siquiera sabe que existe."

CREATE POLICY

USING

WITH CHECK

FORCE RLS

current_setting

SET LOCAL



Próxima clase (Mar 31): Redis — Bases de datos en memoria

Base de Datos Avanzadas · UPCh 2026 · Ing. Ramsés Camas Nájera